LEARNING GUIDE · MINI PROJECT AI

Knight Trap

Implementasi AI pada Game Knight Trap Menggunakan Algoritma Minimax dengan Alpha-Beta Pruning

Louis Yuriel Saputra 0706012410050

Bagianmu: Minimax, Alpha-Beta Pruning, tingkat kesulitan, panel statistik, uji kebenaran Alpha-Beta

Artificial Intelligence · Information Management & Technology · Universitas Ciputra 2026/2027
Disusun dari proposal (v1.1) dan kode di repo KnightTrap · 3 November 2026

Daftar Isi

1. Peta Belajarmu
2. Gambaran Proyek (Proposal Bab 1)
3. Aturan Game → Kode (Proposal Bab 2.1)
4. Alur Satu Giliran di Kode
5. Konsep AI yang Wajib Dipahami Semua Anggota
6. Minimax: Dari Pseudocode ke Kode
7. Alpha-Beta Pruning
8. Langkah, make/unmake, dan Kandidat Jebakan
9. Tiga Pembatasan Agar Cepat
10. Tingkat Kesulitan, Tombol Alpha-Beta, Panel Statistik
11. Pengujian Kebenaran Alpha-Beta
12. Hasil Pengujian (Proposal Bab 2.2)
13. Tambahan di Luar Proposal: Makan dan Waktu Giliran
14. Latihan
15. Pertanyaan Presentasi yang Mungkin Muncul
16. Checklist Sebelum Presentasi

1 Peta Belajarmu

Setelah mempelajari guide ini, kamu bisa:

- Menjelaskan Minimax dan Alpha-Beta Pruning dengan contoh pohon kecil, termasuk kapan cutoff terjadi.
- Menunjuk baris kode `search()` di `js/ai.js` untuk setiap baris pseudocode Gambar 2.1.1.
- Menjelaskan tiga pembatasan: kandidat jebakan maks. 12, pengurutan langkah, dan batas waktu 4 detik.
- Menjelaskan tingkat kesulitan, tombol Alpha-Beta ON/OFF, dan isi panel "Statistik AI".
- Membaca dan membela hasil uji kebenaran Alpha-Beta (30 posisi, depth 1–3).

Tugasmu di Timeline (Bab 2.3)

Minggu	Pekerjaan	Kode
2	Minimax dasar: pohon permainan, batas kedalaman, pilih langkah terbaik	`ai.js`: `search()`, `ab()`, `generateMoves()`, `make()`/`unmake()`
2–3	Alpha-Beta, pengurutan langkah, kandidat jebakan, batas 4 detik, tombol pemangkasan	`ai.js`: `orderMoves()`, `trapCandidates()`, `ABORT`; `main.js`: tombol `#game-ab`
3	Tingkat kesulitan dan panel statistik AI	`config.js`: `DIFFICULTY`; `main.js`: `startAiGame()`, `ui.stats()`
3	Pengujian Minimax vs Alpha-Beta	`tests/run.mjs` bagian 2

Urutan belajar yang disarankan: Bab 2–5 (dasar bersama) → Minimax → Alpha-Beta → Langkah & make/unmake → Tiga pembatasan → Level & panel → Pengujian → Latihan → Pertanyaan presentasi.

2 Gambaran Proyek (Proposal Bab 1)

Apa yang kita buat

Knight Trap adalah varian dari permainan papan *Isolation* (1972). Dua knight ada di papan 7 × 7. Pada gilirannya, pemain:

1. **Lompat**: menggerakkan knight seperti kuda catur (huruf L) ke kotak yang belum terbakar atau dijebak.
2. **Bakar**: kotak yang ditinggalkan langsung terbakar dan tidak bisa dipakai lagi.
3. **Jebak**: menghancurkan satu kotak terbuka lain di mana pun.

Pemain yang **tidak punya lompatan legal** pada gilirannya kalah. Di versi kode, mendarat tepat di kotak knight musuh juga langsung menang (**makan**, tambahan di luar proposal; lihat bab "Tambahan di Luar Proposal"). Game ini dibuat untuk **rally games** (NPLC Universitas Ciputra, tema *Human vs AI*), jadi aturannya harus bisa dijelaskan dalam satu menit, satu game selesai dalam beberapa menit, dan lawan AI-nya tidak boleh terlalu mudah.

Masalah yang diselesaikan (Bab 1.2)

Kita harus membuat lawan AI yang (1) memilih langkah yang kuat, (2) cukup cepat untuk browser, dan (3) punya beberapa tingkat kesulitan. Ada dua tantangan utama:

- **Pohon permainan besar.** Satu langkah = pilihan lompatan (maks. 8) × pilihan kotak jebakan (bisa puluhan). Jumlah posisi meledak setiap giliran.
- **Menilai posisi tanpa main sampai habis.** Kita butuh *fungsi evaluasi* yang memberi skor pada posisi di tengah permainan.

Solusinya, sesuai ketentuan tugas yang membatasi kita ke **empat algoritma**: **Minimax** + **Alpha-Beta Pruning** untuk mengambil keputusan, dan **BFS** + **DFS** sebagai dasar fungsi evaluasi.

Peta file kode

File	Isi	Pemilik utama (Timeline)
`js/config.js`	Konstanta: ukuran papan, posisi awal, kedalaman tiap level, lama satu giliran, batas AI	Shatrya (setup), Louis (AI)
`js/rules.js`	Aturan game: lompatan legal, bakar, jebakan, makan, cek kalah. Tidak menyentuh DOM	Shatrya
`js/ai.js`	Minimax + Alpha-Beta, BFS, DFS, fungsi evaluasi, pemain acak & greedy	Louis (search), Clarrence (BFS), Shatrya (DFS)
`js/ai.worker.js`, `js/ai-client.js`	Menjalankan AI di Web Worker supaya tampilan tidak macet	Louis
`js/game.js`	Alur giliran: input pemain, giliran AI, penghitung waktu giliran, game over	Shatrya & Louis
`js/board.js`, `css/style.css`, `index.html`	Tampilan papan, knight, penanda lompatan, layout	Clarrence
`js/main.js`, `js/i18n.js`	Layar menu, pengaturan, Cara Bermain, panel statistik, teks dua bahasa	Clarrence & Louis
`js/net.js`	Multiplayer online (PeerJS)	Tambahan di luar proposal
`tests/run.mjs`	Semua pengujian Bab 2.2, plus uji makan dan waktu giliran	Louis, Clarrence (pengujian)

> **Repo dengan komentar per baris.** Folder `KnightTrap-Annotated` berisi kode yang sama persis, tetapi setiap baris diberi komentar bahasa Indonesia. Pakai repo itu untuk belajar; potongan kode di guide ini diambil dari repo asli `KnightTrap` supaya nomor barisnya cocok dengan kode yang dijalankan.

3 Aturan Game → Kode (Proposal Bab 2.1)

Setiap aturan di proposal punya tempat yang jelas di `js/rules.js`. File ini adalah **satu-satunya sumber aturan**: tampilan, AI, dan mode online semuanya memakai fungsi yang sama.

Aturan proposal	Kode
Papan 7 × 7, array 49 kotak (kosong / terbakar / jebakan) + posisi 2 knight	`createState()`, konstanta `EMPTY`, `BURNED`, `TRAP`
(a) Knight bergerak huruf L	`JUMPS` dan `NEIGHBORS` (dihitung sekali di awal)
(b) Kotak yang ditinggalkan terbakar	`applyJump()`
(c) Hancurkan satu kotak terbuka	`applyTrap()`; validasi di `validateMove()`
(d) Tidak boleh mendarat di kotak terbakar atau jebakan. Proposal juga melarang kotak lawan; di kode, mendarat di sana = makan (tambahan)	`legalJumps()` (cek `cells[n] === EMPTY`); `isOpen()` untuk kotak jebakan
(e) Tidak punya lompatan legal = kalah	`winner()` memakai `canMove()`
Tambahan: dimakan = kalah, langkah makan tanpa jebakan	`winner()` (`pos[0] === pos[1]`), `validateMove()`

Representasi papan

Kotak diberi nomor 0 sampai 48: `index = baris × 7 + kolom`, baris 0 adalah baris paling atas (rank 7). Jadi **a1 = 42** (kiri bawah) dan **g7 = 6** (kanan atas).

js/config.js : baris 3–8

```
3  export const N = 7;
4  export const SIZE = N * N;
5
6  // The proposal does not fix start squares. P1 starts at a1 (bottom-left),
7  // P2 at g7 (top-right). Index = row * 7 + col, row 0 is the top rank (7).
8  export const START = Object.freeze({ p1: 42, p2: 6, first: 0 });
```

js/rules.js : baris 28–35

```
28 export function createState(start = START) {
29   return {
30     cells: new Int8Array(SIZE),
31     pos: [start.p1, start.p2],
32     turn: start.first ?? 0,
33     moves: 0,
34   };
35 }
```

`cells` memakai `Int8Array` (array angka 8-bit) karena kecil dan cepat disalin. `pos[0]` adalah posisi knight P1, `pos[1]` knight P2, `turn` adalah siapa yang jalan (0 atau 1).

Tetangga knight dihitung sekali

js/rules.js : baris 10–26

```
const JUMPS = [
  [-2, -1], [-2, 1], [-1, -2], [-1, 2],
  [1, -2], [1, 2], [2, -1], [2, 1],
];

// Knight neighbor lists, computed once.
export const NEIGHBORS = Array.from({ length: SIZE }, (_, i) => {
  const r = Math.floor(i / N);
  const c = i % N;
  const out = [];
  for (const [dr, dc] of JUMPS) {
    const rr = r + dr;
    const cc = c + dc;
    if (rr >= 0 && rr < N && cc >= 0 && cc < N) out.push(rr * N + cc);
  }
  return out;
});
```

Untuk setiap kotak, kita coba 8 arah L dan simpan hanya yang masih di dalam papan. Hasilnya tabel 49 daftar. AI memanggil daftar ini jutaan kali, jadi menghitungnya sekali di awal menghemat banyak waktu.

Satu giliran = lompat, bakar, jebak

js/rules.js : baris 54–56

```
export function isOpen(s, sq) {
  return s.cells[sq] === EMPTY && sq !== s.pos[0] && sq !== s.pos[1];
}
```

js/rules.js : baris 60–64

```
export function legalJumps(s, side = s.turn) {
  const out = [];
  for (const n of NEIGHBORS[s.pos[side]]) if (s.cells[n] === EMPTY) out.push(n);
  return out;
}
```

js/rules.js : baris 77–83

```
export function applyJump(s, to) {
  const side = s.turn;
  const from = s.pos[side];
  s.cells[from] = BURNED;
  s.pos[side] = to;
  return from;
}
```

js/rules.js : baris 86–88

```
export function applyTrap(s, sq) {
  if (sq >= 0) s.cells[sq] = TRAP;
}
```

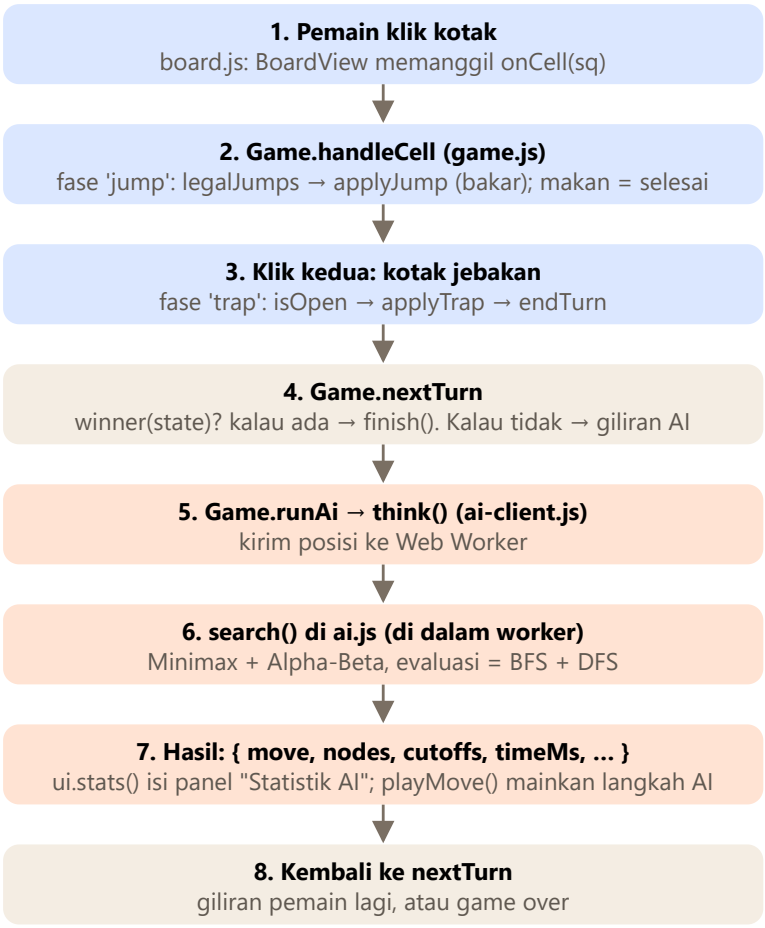
js/rules.js : baris 116–119

```
116  export function winner(s) {
117    if (s.pos[0] === s.pos[1]) return s.turn ^ 1;
118    return canMove(s, s.turn) ? null : s.turn ^ 1;
119  }
```

Kenapa `winner()` mengembalikan `s.turn ^ 1`? `^` adalah XOR. `0 ^ 1 = 1` dan `1 ^ 1 = 0`, jadi ini cara singkat menulis "pemain lainnya". Kalau yang sedang jalan tidak bisa bergerak, atau baru saja dimakan (kedua knight di kotak yang sama), lawannya menang.

4 Alur Satu Giliran di Kode

Bagian ini menghubungkan semua file. Setiap anggota harus bisa menceritakan alur ini saat presentasi.



Gambar: alur satu ronde (pemain lalu AI). Biru = input pemain, krem = alur game, oranye = AI.

js/game.js : baris 127–146

```
127   nextTurn() {
128     if (this.disposed) return;
129     const w = winner(this.state);
130     if (w !== null) return this.finish(w);
131     const p = this.current;
132     if (p.kind === 'human') {
133       this.phase = 'jump';
134       this.startClock();
135       this.refresh();
136     } else if (p.kind === 'ai') {
137       this.phase = 'ai';
138       this.stopClock();
139       this.refresh();
140       this.runAi();
141     } else {
142       this.phase = 'remote';
143       this.startClock();
144       this.refresh();
145     }
146   }
```

js/game.js : baris 209–233

```
  async runAi() {
    this.ui.thinking(true);
    const t0 = performance.now();
    let res = null;
    try {
      res = await think(this.state, { depth: this.ai.depth, prune: this.ai.prune });
    } catch (err) {
      console.error('AI failed', err);
    }
    if (this.disposed) return;
    const elapsed = performance.now() - t0;
    if (elapsed < AICFG.minUiDelayMs) await sleep(AICFG.minUiDelayMs - elapsed);
    if (this.disposed) return;
    this.ui.thinking(false);
    let move = res?.move;
    if (!move || !validateMove(this.state, move)) {
      console.warn('AI returned no valid move; falling back to a random legal move');
      move = randomMove(this.state);
    } else {
      this.ui.stats(res);
    }
    await this.playMove(move);
    if (this.disposed) return;
    this.nextTurn();
  }
```

Perhatikan dua pengaman di `runAi`: (1) kalau AI terlalu cepat (Easy), tampilan menunggu minimal 450 ms supaya tidak terasa "langsung"; (2) kalau AI gagal atau mengembalikan langkah tidak sah, game memakai langkah acak yang legal, jadi game tidak pernah macet.

js/ai-client.js : baris 44–53

```
export function think(state, { depth, prune }) {
  const args = { state: serialize(state), depth, prune };
  const w = getWorker();
  if (!w) return runInline(args);
  return new Promise((resolve, reject) => {
    const id = ++seq;
    pending.set(id, { resolve, reject, args });
    w.postMessage({ id, ...args });
  });
}
```

js/ai.worker.js : baris 5–13

```
self.onmessage = (e) => {
  const { id, state, depth, prune } = e.data;
  try {
    const result = search(deserialize(state), { depth, prune });
    self.postMessage({ id, result });
  } catch (err) {
    self.postMessage({ id, error: String(err && err.message ? err.message : err) });
  }
};
```

Kenapa Web Worker? JavaScript di browser berjalan di satu thread. Kalau Expert berpikir 1–2 detik di thread utama, tombol dan penghitung waktu akan membeku. Worker menjalankan `search()` di thread terpisah, jadi papan tetap hidup saat AI berpikir.

5 Konsep AI yang Wajib Dipahami Semua Anggota

Dosen bisa bertanya ke siapa saja. Bagian ini adalah dasar yang harus dikuasai semua anggota, walaupun detailnya milik anggota lain.

Agen dan model P.E.A.S (Tabel 2.1.1)

Komponen	Di Knight Trap	Di kode
Performance	Win rate AI, waktu berpikir ≤ 4 detik, jumlah posisi diperiksa sedikit, tantangan pas untuk siswa SMA	`tests/run.mjs`, panel statistik
Environment	Papan 7 × 7, dua agen bergiliran. Deterministik, fully observable, diskret, berurutan, zero-sum	`createState()`
Actuators	Lompat ke kotak legal (maks. 8 arah) + hancurkan satu kotak	`applyMove()`, `{ jump, trap }`
Sensors	Posisi kedua knight, status 49 kotak, giliran	`state.cells`, `state.pos`, `state.turn`

Kenapa sifat lingkungan penting? Minimax hanya cocok untuk game *deterministik* (tidak ada dadu), *fully observable* (tidak ada kartu tertutup), dan *zero-sum* (untung saya = rugi lawan). Knight Trap memenuhi ketiganya.

Adversarial search dan game tree

Game tree: akar = posisi sekarang, cabang = langkah, simpul = posisi hasil langkah. **Kedalaman** (depth) = berapa giliran ke depan yang dilihat AI. **Branching factor** = rata-rata jumlah langkah per posisi. Di Knight Trap: sampai 8 lompatan × sampai 12 kandidat jebakan = sampai **96 langkah** per posisi. Dengan depth 4: $96^4 \approx 85$ juta posisi. Itulah alasan kita butuh Alpha-Beta dan pembatasan kandidat.

Minimax dalam satu paragraf

AI adalah **MAX** (ingin skor setinggi mungkin), manusia adalah **MIN** (dianggap selalu memilih langkah terburuk untuk AI). AI menelusuri pohon sampai kedalaman tertentu, menilai posisi di ujung dengan fungsi evaluasi, lalu menaikkan nilai ke atas: di simpul MAX ambil nilai terbesar, di simpul MIN ambil nilai terkecil. Langkah di akar dengan nilai terbesar dipilih.

Alpha-Beta dalam satu paragraf

Alpha = nilai terbaik yang sudah pasti didapat MAX. **Beta** = nilai terbaik yang sudah pasti didapat MIN. Kalau di suatu simpul `beta ≤ alpha`, sisa langkah di simpul itu tidak mungkin mengubah keputusan, jadi dilewati (**cutoff**). Hasilnya **sama persis** dengan Minimax biasa, tapi posisi yang diperiksa jauh lebih sedikit. Ini dibuktikan di pengujian.

BFS dan DFS dalam satu paragraf

BFS (antrian, menyebar per "lapisan") menghitung jumlah lompatan minimum dari knight ke setiap kotak. Kotak yang lebih cepat dicapai knight A adalah **wilayah** A. **DFS** (tumpukan, masuk sedalam mungkin dulu)

melakukan **flood fill**: menandai semua kotak yang bisa dicapai knight. Kalau area kedua knight tidak bersentuhan, papan sudah **terbelah** dan pemenangnya hampir pasti yang areanya lebih luas.

js/ai.js : baris 80–115

```
80  export function evaluate(s, me, ply = 0) {
81    const { cells, pos, turn } = s;
82    const sign = turn === me ? 1 : -1;
83    // The side to move was just eaten, or has no jump: it has lost.
84    if (pos[0] === pos[1] || !hasJump(s, turn)) return -sign * (WIN - ply);
85    // The side to move can eat right now, so it wins on the next ply.
86    if (NEIGHBORS[pos[turn]].includes(pos[turn ^ 1])) return sign * (WIN - ply - 1);
87    const areaA = dfs(cells, pos, 0, markA);
88    const areaB = dfs(cells, pos, 1, markB);
89    let split = true;
90    for (let i = 0; i < SIZE; i++) {
91      if (markA[i] === 1 && markB[i] === 1) {
92        split = false;
93        break;
94      }
95    }
96    let score;
97    if (split) {
98      score = (areaA - areaB) * SPLIT_WEIGHT;
99    } else {
100     bfs(cells, pos, 0, distA);
101     bfs(cells, pos, 1, distB);
102     let terr = 0;
103     for (let i = 0; i < SIZE; i++) {
104       if (!open(cells, pos, i)) continue;
105       const a = distA[i];
106       const b = distB[i];
107       if (a < 0 && b < 0) continue;
108       if (b < 0 || (a >= 0 && a < b)) terr++;
109       else if (a < 0 || b < a) terr--;
110       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
111     }
112     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
113   }
114   return me === 0 ? score : -score;
115 }
```

Inti fungsi evaluasi (skor dari sudut pandang P1, lalu dibalik tandanya untuk P2):

- Cek dulu posisi yang sudah pasti: yang jalan sudah dimakan atau tidak bisa lompat = kalah (`WIN − ply`); yang jalan bisa langsung makan = menang di ply berikutnya (`WIN − ply − 1`).
- Kalau papan sudah terbelah (DFS): `skor = (areaP1 − areaP2) × 100` .
- Kalau belum: `skor = selisih wilayah (BFS) + 0,5 × selisih mobilitas` . Mobilitas = jumlah lompatan legal saat ini.

Angka 100 (`SPLIT_WEIGHT`) dipilih karena skor wilayah paling besar sekitar 53, jadi papan yang sudah terbelah selalu dinilai lebih "pasti" daripada posisi yang masih terbuka.

6 Minimax: Dari Pseudocode ke Kode

Ide dasarnya

AI tidak tahu langkah mana yang akan dipilih manusia, jadi AI menganggap manusia **selalu memilih langkah terbaik untuk dirinya sendiri** (paling buruk untuk AI). Dengan anggapan itu, AI memilih langkah yang **nilai terburuknya paling baik**. Itulah "min" dan "max".

> **Contoh dengan tangan**
>
> AI (MAX) punya dua langkah, A dan B. Setelah setiap langkah, manusia (MIN) punya dua balasan. Skor di ujung dari fungsi evaluasi:
>
> - Langkah A → balasan menghasilkan skor 3 atau 5. MIN memilih **3**.
> - Langkah B → balasan menghasilkan skor 2 atau 9. MIN memilih **2**.
>
> MAX membandingkan 3 dan 2, lalu memilih **A** (nilai 3). Walaupun B punya skor 9, manusia yang pintar tidak akan memberikannya.

Pemetaan pseudocode Gambar 2.1.1 → kode

Pseudocode	`ab()` di `js/ai.js`
`if player has no legal jump then return terminal score`	`if (!hasJump(s, side)) return side === aiSide ? -(WIN - ply) : WIN - ply;`
(tambahan, tidak ada di pseudocode) pemain baru dimakan	`if (s.pos[0] === s.pos[1]) return side === aiSide ? -(WIN - ply) : WIN - ply;`
`if depth = 0 then return EVALUATE(state)`	`if (d === 0) return evaluate(s, aiSide, ply);` (`ply` ikut dikirim supaya skor menang/kalah di ujung sama dengan skor dari pencarian)
`moves ← GENERATE(state, player)`	`const moves = generateMoves(s, side);`
`moves ← ORDER(moves)`	`if (d >= CFG.orderMinDepth) orderMoves(...)`
`if player = AI then … best ← max(best, v); alpha ← max(alpha, best)`	blok `if (isMax) { … }`
`else … best ← min(best, v); beta ← min(beta, best)`	blok setelahnya (MIN)
`if beta ≤ alpha then break`	`if (prune && beta <= alpha) { cutoffs++; break; }`
`APPLY(state, move)`	`make(s, m)` lalu `unmake(s, m, from)` setelah rekursi

`js/ai.js : baris 219–309`

```
219  export function search(state, opts = {}) {
220    const depth = Math.max(1, opts.depth ?? 2);
221    const prune = opts.prune ?? true;
```

```
  const timeLimitMs = opts.timeLimitMs ?? CFG.timeLimitMs;
  const now = opts.now ?? defaultNow;
  const s = cloneState(state);
  const aiSide = s.turn;
  const t0 = now();
  const deadline = t0 + timeLimitMs;
  let nodes = 1;
  let cutoffs = 0;

  function ab(d, alpha, beta, ply) {
    nodes++;
    if ((nodes & 255) === 0 && now() > deadline) throw ABORT;
    const side = s.turn;
    if (s.pos[0] === s.pos[1]) return side === aiSide ? -(WIN - ply) : WIN - ply;
    if (!hasJump(s, side)) return side === aiSide ? -(WIN - ply) : WIN - ply;
    if (d === 0) return evaluate(s, aiSide, ply);
    const moves = generateMoves(s, side);
    const isMax = side === aiSide;
    if (d >= CFG.orderMinDepth) orderMoves(s, moves, aiSide, isMax);
    if (isMax) {
      let best = -Infinity;
      for (const m of moves) {
        const from = make(s, m);
        const v = ab(d - 1, alpha, beta, ply + 1);
        unmake(s, m, from);
        if (v > best) best = v;
        if (best > alpha) alpha = best;
        if (prune && beta <= alpha) {
          cutoffs++;
          break;
        }
      }
      return best;
    }
    let best = Infinity;
    for (const m of moves) {
      const from = make(s, m);
      const v = ab(d - 1, alpha, beta, ply + 1);
      unmake(s, m, from);
      if (v < best) best = v;
      if (best < beta) beta = best;
      if (prune && beta <= alpha) {
        cutoffs++;
        break;
      }
    }
    return best;
  }

  if (!hasJump(s, aiSide)) return null;
  const moves = generateMoves(s, aiSide);
  if (depth >= CFG.orderMinDepth) orderMoves(s, moves, aiSide, true);

  let best = -Infinity;
  let bestMove = moves[0];
  let alpha = -Infinity;
  let completed = 0;
  let timedOut = false;
  try {
    for (const m of moves) {
      const from = make(s, m);
      const v = ab(depth - 1, alpha, Infinity, 1);
      unmake(s, m, from);
```

```
      completed++;
      if (v > best) {
        best = v;
        bestMove = m;
      }
      if (prune && best > alpha) alpha = best;
    }
  } catch (e) {
    if (e !== ABORT) throw e;
    timedOut = true;
  }

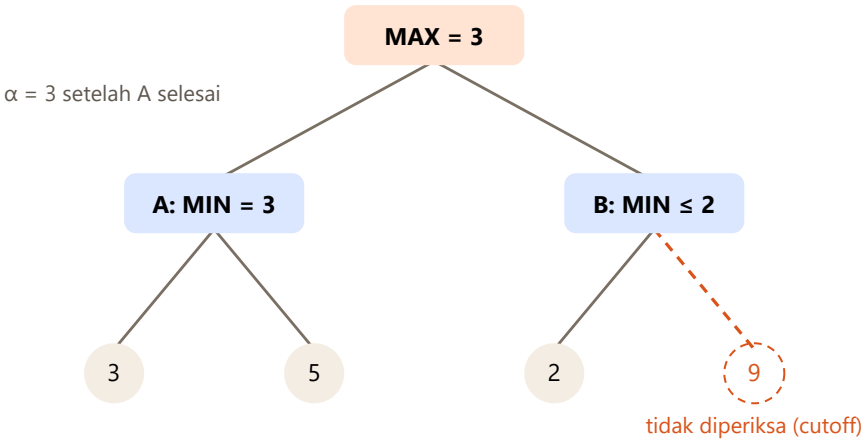
  return {
    move: unpack(bestMove),
    score: completed > 0 ? best : null,
    depth,
    prune,
    nodes,
    cutoffs,
    timeMs: now() - t0,
    timedOut,
    rootMoves: moves.length,
    completedRootMoves: completed,
  };
}
```

Penjelasan bagian penting

- **Baris awal:** `depth` minimal 1, `prune` default ON, batas waktu default 4000 ms. `cloneState` membuat salinan supaya posisi di layar tidak berubah selama AI mencoba langkah.
- **`aiSide = s.turn`:** siapa pun yang sedang jalan menjadi MAX. Jadi fungsi yang sama juga bisa dipakai untuk menguji AI melawan AI.
- **`nodes` dan `cutoffs`:** penghitung untuk panel statistik dan pengujian. Setiap panggilan `ab()` = satu node.
- **Skor menang `WIN - ply`:** `WIN` = 1.000.000. Dikurangi `ply` (jarak dari akar) supaya menang dalam 2 giliran dinilai lebih tinggi daripada menang dalam 4 giliran. AI jadi memilih kemenangan tercepat, dan kalau kalah, menunda kekalahan selama mungkin.
- **Akar ditangani terpisah** (bukan lewat `ab()`) karena di akar kita perlu tahu *langkah mana* yang terbaik, bukan hanya nilainya. `bestMove` diperbarui setiap ada nilai lebih tinggi.
- **Hasil:** objek berisi `move`, `score`, `depth`, `prune`, `nodes`, `cutoffs`, `timeMs`, `timedOut`. Persis sesuai *Expected Output* di proposal.

7 Alpha-Beta Pruning

Contoh yang sama, dengan pemangkasan



Setelah cabang A, MAX sudah dijamin 3 (alpha = 3). Di B, balasan pertama bernilai 2, jadi MIN pasti memberi ≤ 2. Karena 2 ≤ 3, MAX tidak akan memilih B, dan daun 9 tidak perlu dilihat.

Langkah	Simpul	α	β	Yang terjadi
1	A, daun 3	−∞	3	MIN: best = 3, beta = 3
2	A, daun 5	−∞	3	5 tidak lebih kecil dari 3, A = 3
3	Akar	3	+∞	MAX: best = 3, alpha = 3
4	B, daun 2	3	2	beta = 2 ≤ alpha = 3 → **cutoff**, daun 9 dilewati
5	Akar	3	+∞	B bernilai ≤ 2, A tetap terbaik → pilih A

> **Kalimat kunci untuk presentasi:** "Alpha-Beta tidak menebak. Cabang yang dibuang adalah cabang yang *terbukti* tidak bisa mengubah keputusan, jadi hasilnya selalu sama dengan Minimax biasa. Kami membuktikannya di 30 posisi pada depth 1 sampai 3."

Kode bagian MAX dan MIN

js/ai.js : baris 231–269

```
231   function ab(d, alpha, beta, ply) {
232     nodes++;
233     if ((nodes & 255) === 0 && now() > deadline) throw ABORT;
234     const side = s.turn;
235     if (s.pos[0] === s.pos[1]) return side === aiSide ? -(WIN - ply) : WIN - ply;
236     if (!hasJump(s, side)) return side === aiSide ? -(WIN - ply) : WIN - ply;
237     if (d === 0) return evaluate(s, aiSide, ply);
238     const moves = generateMoves(s, side);
239     const isMax = side === aiSide;
240     if (d >= CFG.orderMinDepth) orderMoves(s, moves, aiSide, isMax);
241     if (isMax) {
242       let best = -Infinity;
243       for (const m of moves) {
244         const from = make(s, m);
```

```
        const v = ab(d - 1, alpha, beta, ply + 1);
        unmake(s, m, from);
        if (v > best) best = v;
        if (best > alpha) alpha = best;
        if (prune && beta <= alpha) {
          cutoffs++;
          break;
        }
      }
      return best;
    }
    let best = Infinity;
    for (const m of moves) {
      const from = make(s, m);
      const v = ab(d - 1, alpha, beta, ply + 1);
      unmake(s, m, from);
      if (v < best) best = v;
      if (best < beta) beta = best;
      if (prune && beta <= alpha) {
        cutoffs++;
        break;
      }
    }
    return best;
  }
```

Perhatikan `if (prune && …)`: kalau tombol Alpha-Beta OFF, `prune` bernilai `false` dan loop tidak pernah berhenti lebih awal, jadi kode yang sama berubah menjadi Minimax biasa. Itulah kenapa perbandingannya adil: **satu-satunya** perbedaan adalah pemangkasan.

Di akar, alpha juga diperbarui (`if (prune && best > alpha) alpha = best;`) lalu diteruskan ke setiap anak, sedangkan beta di akar selalu `Infinity` karena tidak ada simpul MIN di atas akar.

8 Langkah, make/unmake, dan Kandidat Jebakan

Satu langkah dikemas dalam satu angka

js/ai.js : baris 149–151

```
149  // A move is packed as jump | (trap + 1) << 6, so trap -1 (an eating move, or no square left) packs to
0.
150  export const pack = (jump, trap) => jump | ((trap + 1) << 6);
151  export const unpack = (m) => ({ jump: m & 63, trap: (m >> 6) - 1 });
```

Kotak lompatan 0–48 muat dalam 6 bit (maks. 63). Kotak jebakan + 1 disimpan di bit di atasnya, sehingga jebakan −1 (langkah makan, atau tidak ada kotak tersisa) menjadi 0. Contoh: lompat ke 30, jebak 12 → `30 | (13 << 6) = 30 + 832 = 862`. Satu angka lebih hemat memori daripada objek `{ jump, trap }`, dan AI membuat ribuan langkah per detik.

js/ai.js : baris 154–173

```
154  export function generateMoves(s, side, out = []) {
155    const { cells, pos } = s;
156    const from = pos[side];
157    const opp = side ^ 1;
158    for (const j of NEIGHBORS[from]) {
159      if (cells[j] !== EMPTY) continue;
160      if (j === pos[opp]) {
161        out.push(pack(j, -1));
162        continue;
163      }
164      cells[from] = BURNED;
165      pos[side] = j;
166      trapCandidates(s, opp, trapBuf);
167      if (trapBuf.length === 0) out.push(pack(j, -1));
168      else for (const t of trapBuf) out.push(pack(j, t));
169      pos[side] = from;
170      cells[from] = EMPTY;
171    }
172    return out;
173  }
```

Untuk setiap lompatan legal, knight dipindahkan sementara (kotak asal dibakar) supaya kandidat jebakan dihitung dari posisi *setelah* lompatan, lalu semuanya dikembalikan. Ini persis aturan: jebak dilakukan setelah lompat. Kalau tetangga itu adalah kotak knight musuh, yang dibuat hanya satu langkah makan, `pack(j, -1)`, karena game langsung selesai.

make / unmake: coba lalu batalkan tanpa menyalin papan

js/ai.js : baris 175–192

```
function make(s, m) {
  const side = s.turn;
  const from = s.pos[side];
  const t = (m >> 6) - 1;
  s.cells[from] = BURNED;
  s.pos[side] = m & 63;
  if (t >= 0) s.cells[t] = TRAP;
  s.turn ^= 1;
  return from;
}

function unmake(s, m, from) {
  s.turn ^= 1;
  const t = (m >> 6) - 1;
  if (t >= 0) s.cells[t] = EMPTY;
  s.pos[s.turn] = from;
  s.cells[from] = EMPTY;
}
```

Proposal: "langkah dicoba dan dibatalkan langsung pada papan yang sama tanpa menyalin papan, supaya cepat". `make` mengembalikan `from` (kotak asal) karena itu satu-satunya informasi yang hilang dan dibutuhkan `unmake`. Urutan di `unmake` adalah kebalikan `make`.

9 Tiga Pembatasan Agar Cepat

1. Kandidat jebakan maksimal 12

js/ai.js : baris 119–147

```
function trapCandidates(s, opp, out) {
  const { cells, pos } = s;
  const max = CFG.maxTrapCandidates;
  out.length = 0;
  stampId++;
  const ring1 = NEIGHBORS[pos[opp]];
  for (const n of ring1) {
    if (open(cells, pos, n)) {
      stamp[n] = stampId;
      out.push(n);
    }
  }
  for (const n of ring1) {
    if (out.length >= max) break;
    if (!open(cells, pos, n)) continue;
    for (const m of NEIGHBORS[n]) {
      if (out.length >= max) break;
      if (stamp[m] !== stampId && open(cells, pos, m)) {
        stamp[m] = stampId;
        out.push(m);
      }
    }
  }
  // The opponent is boxed in already: any open square is a legal trap.
  if (out.length === 0) {
    for (let i = 0; i < SIZE && out.length < max; i++) if (open(cells, pos, i)) out.push(i);
  }
  return out;
}
```

Tanpa batas, satu lompatan bisa diikuti ±45 pilihan jebakan. Jebakan yang paling berguna biasanya di dekat lawan, jadi urutannya: (1) kotak tujuan lompatan lawan (ring 1), (2) kotak dua lompatan dari lawan (ring 2), berhenti di 12. `stamp` / `stampId` adalah trik untuk menandai "sudah dimasukkan" tanpa mengosongkan array setiap kali: cukup naikkan `stampId`. Kalau lawan sudah terkurung (tidak ada kotak di ring 1/2), kotak terbuka apa pun dipakai.

2. Pengurutan langkah (move ordering)

js/ai.js : baris 201–212

```
export function orderMoves(s, moves, aiSide, maximizing) {
  const enemy = s.pos[s.turn ^ 1];
  const scored = moves.map((m) => {
    if ((m & 63) === enemy) return { m, v: maximizing ? Infinity : -Infinity };
    const from = make(s, m);
    const v = evaluate(s, aiSide);
    unmake(s, m, from);
    return { m, v };
  });
  scored.sort(maximizing ? (a, b) => b.v - a.v : (a, b) => a.v - b.v);
  for (let i = 0; i < moves.length; i++) moves[i] = scored[i].m;
}
```

Alpha-Beta memangkas paling banyak kalau langkah terbaik diperiksa **lebih dulu**. Langkah makan langsung diberi nilai paling ekstrem (`±Infinity`) supaya selalu diperiksa pertama, karena langkah itu mengakhiri game. Langkah lain dinilai sekali dengan `evaluate()` (evaluasi statis), lalu diurutkan: MAX dari besar ke kecil, MIN dari kecil ke besar. Pengurutan sendiri butuh waktu, jadi hanya dilakukan kalau sisa kedalaman ≥ 2 (`orderMinDepth`), sesuai proposal.

3. Batas waktu 4 detik

js/ai.js : baris 231–233

```
  function ab(d, alpha, beta, ply) {
    nodes++;
    if ((nodes & 255) === 0 && now() > deadline) throw ABORT;
```

Setiap 256 node (`(nodes & 255) === 0`, lebih murah daripada mengecek jam di setiap node), waktu dicek. Kalau melewati batas, dilempar `ABORT`. Di akar, `try/catch` menangkapnya dan AI memakai `bestMove` dari langkah akar yang sudah selesai dihitung (`completed`). `timedOut: true` lalu muncul sebagai label "batas 4 dtk" di panel. Pada hasil uji, Expert maksimum 696 ms, jadi batas ini adalah pengaman, bukan sesuatu yang sering terjadi.

10 Tingkat Kesulitan, Tombol Alpha-Beta, Panel Statistik

js/config.js : baris 10–29

```
10  export const DIFFICULTY = Object.freeze({
11    easy: 1,
12    medium: 2,
13    hard: 3,
14    expert: 4,
15  });
16
17  // Longest a player may take for one turn, the jump and the trap together. Change the length here.
18  export const TURN_SECONDS = 15;
19  // How long past the limit the online host waits for the guest's own move before playing one for it.
20  export const TURN_GRACE_MS = 1500;
21
22  export const AI = Object.freeze({
23    timeLimitMs: 4000,
24    maxTrapCandidates: 12,
25    orderMinDepth: 2,
26    // Shortest visible "thinking" time so instant replies at Easy do not feel jarring.
27    // UI only; not counted in the reported think time.
28    minUiDelayMs: 450,
29  });
```

Level hanya mengubah **kedalaman**. Easy = 1 (AI hanya melihat langkahnya sendiri), Expert = 4 (AI, manusia, AI, manusia).

js/main.js : baris 262–271

```
262  function startAiGame() {
263    const humanFirst = settings.first === 'human' || (settings.first === 'random' && Math.random() <
0.5);
264    const hs = humanFirst ? 0 : 1;
265    const players = [];
266    players[hs] = { kind: 'human', name: t('you'), sub: colorName(hs) };
267    players[hs ^ 1] = { kind: 'ai', name: `AI · ${t(settings.diff)}`, sub: colorName(hs ^ 1) };
268    $('#game-ab').checked = settings.prune;
269    $('#ab-state').textContent = settings.prune ? 'ON' : 'OFF';
270    launch({ mode: 'ai', level: settings.diff, players, ai: { depth: DIFFICULTY[settings.diff], prune:
settings.prune } });
271  }
```

js/main.js : baris 170–186

```
170    stats(res) {
171      const set = (id, html) => {
172        $(id).innerHTML = html;
173      };
174      set('#st-depth', String(res.depth));
175      set('#st-nodes', res.nodes.toLocaleString());
176      set('#st-cutoffs', res.prune ? res.cutoffs.toLocaleString() : '0 <span class="tag">OFF</span>');
177      const ms = res.timeMs < 10 ? res.timeMs.toFixed(1) : Math.round(res.timeMs).toLocaleString();
178      set('#st-time', `${ms} ms${res.timedOut ? ` <span class="tag">${t('timedOut')}</span>` : ''}`);
179      let score = '–';
180      if (res.score != null) {
181        if (res.score >= WIN / 2) score = t('scoreWin', WIN - res.score);
182        else if (res.score <= -WIN / 2) score = t('scoreLoss', WIN + res.score);
183        else score = (res.score > 0 ? '+' : '') + (Number.isInteger(res.score) ? res.score :
res.score.toFixed(1));
184      }
185      set('#st-score', score);
186    },
```

Panel	Isi	Dari
Kedalaman	Depth pencarian	`res.depth`
Posisi diperiksa	Jumlah node	`res.nodes`
Cabang dipangkas	Jumlah cutoff (0 + "OFF" kalau Alpha-Beta mati)	`res.cutoffs`
Waktu	Lama berpikir (+ "batas 4 dtk" kalau habis waktu)	`res.timeMs`, `res.timedOut`
Skor langkah	Nilai evaluasi, atau "Menang/Kalah dalam n giliran" kalau skor ≥ WIN/2	`res.score`

> **Demo yang meyakinkan:** main di Hard, catat "Posisi diperiksa". Matikan tombol Alpha-Beta di panel, main satu langkah lagi: angka posisi diperiksa melonjak dan waktunya naik, tetapi gaya bermain AI tidak berubah.

11 Pengujian Kebenaran Alpha-Beta

tests/run.mjs : baris 175–210

```
175  // ---------------------------------------------------------------------------
176  console.log('\n2. Alpha-Beta correctness (Alpha-Beta vs plain Minimax, same eval/order, no time
limit)');
177  const positions = [];
178  {
179    let seed = BASE_SEED;
180    while (positions.length < 30) {
181      const rng = mulberry32(seed);
182      const turns = 4 + Math.floor(rng() * 5);
183      const s = createState();
184      let ok = true;
185      for (let i = 0; i < turns; i++) {
186        const m = winner(s) === null ? randomMove(s, rng) : null;
187        if (!m) { ok = false; break; }
188        applyMove(s, m);
189      }
190      if (ok && winner(s) === null) positions.push({ seed, turns, s });
191      seed++;
192    }
193    console.log(`   30 positions from seeds ${positions[0].seed}..${positions.at(-1).seed} (seed
${BASE_SEED} + k, random play of 4-8 turns)`);
194    console.log('   depth | identical score | identical move | nodes Minimax | nodes Alpha-Beta | saved |
cutoffs | ms MM | ms AB');
195    for (const depth of [1, 2, 3]) {
196      let sameScore = 0, sameMove = 0, nMM = 0, nAB = 0, cut = 0, tMM = 0, tAB = 0;
197      for (const p of positions) {
198        const mm = search(p.s, { depth, prune: false, timeLimitMs: Infinity, now:
performance.now.bind(performance) });
199        const ab = search(p.s, { depth, prune: true, timeLimitMs: Infinity, now:
performance.now.bind(performance) });
200        if (mm.score === ab.score) sameScore++;
201        else console.log(`   mismatch seed ${p.seed} depth ${depth}: MM ${mm.score} AB ${ab.score}`);
202        if (mm.move.jump === ab.move.jump && mm.move.trap === ab.move.trap) sameMove++;
203        nMM += mm.nodes; nAB += ab.nodes; cut += ab.cutoffs; tMM += mm.timeMs; tAB += ab.timeMs;
204      }
205      const saved = (100 * (1 - nAB / nMM)).toFixed(1) + '%';
206      console.log(`   ${pad(depth, 5)} | ${pad(sameScore + '/30', 15)} | ${pad(sameMove + '/30', 13)} |
${pad(nMM, 13)} | ${pad(nAB, 16)} | ${pad(saved, 5)} | ${pad(cut, 7)} | ${pad(tMM.toFixed(0), 5)} |
${pad(tAB.toFixed(0), 5)}`);
207      check(`depth ${depth}: identical scores`, sameScore === 30);
208      check(`depth ${depth}: identical moves`, sameMove === 30);
209    }
210  }
```

- **Posisi uji bisa diulang:** `mulberry32(seed)` adalah generator angka acak dengan seed, jadi seed yang sama selalu menghasilkan 30 posisi yang sama.
- **Adil:** keduanya memakai `search()` yang sama dengan `timeLimitMs: Infinity`; hanya `prune` yang berbeda.
- **Lulus** kalau skor dan langkah identik di 30/30 posisi pada setiap depth.

12 Hasil Pengujian (Proposal Bab 2.2)

Semua angka di bawah berasal dari `node tests/run.mjs` (run penuh: 100 game per level per lawan). Hasil akhir run: **900 cek lulus, 0 gagal**. Jalankan sendiri sebelum presentasi supaya kamu tahu prosesnya: `node tests/run.mjs --quick` (±30 detik) atau tanpa `--quick` (±5 menit).

1. Kebenaran Alpha-Beta (Tabel 2.2.3, baris 1)

30 posisi uji dibuat dari permainan acak 4–8 giliran dengan seed `20261028 + k` (bisa diulang siapa pun). Alpha-Beta dan Minimax biasa memakai evaluasi, urutan langkah, dan tanpa batas waktu yang sama; bedanya hanya pemangkasan.

Depth	Skor identik	Langkah identik	Node Minimax	Node Alpha-Beta	Hemat	Cutoff	ms Minimax	ms Alpha-Beta
1	30/30	30/30	981	981	0,0%	0	7	5
2	30/30	30/30	34.035	8.888	73,9%	896	120	32
3	30/30	30/30	999.825	54.465	94,6%	3.001	3.042	281

Cara membaca: di depth 3, Alpha-Beta hanya memeriksa sekitar 1 dari 18 posisi yang diperiksa Minimax biasa, ±11× lebih cepat, dan tetap memilih skor dan langkah yang sama di 30 dari 30 posisi. Di depth 1 tidak ada penghematan karena belum ada simpul MIN yang bisa dipangkas (AI hanya melihat langkahnya sendiri).

2. Kekuatan AI / win rate (Tabel 2.2.3, baris 2)

Level	Depth	vs pemain acak	vs pemain greedy
Easy	1	99%	89%
Medium	2	99%	100%
Hard	3	100%	79%
Expert	4	100%	92%

Target proposal "Medium ke atas menang minimal 90% melawan pemain acak" **tercapai** (99–100%). Melawan greedy, win rate **tidak naik seiring level**: Medium 100%, tetapi Hard hanya 79%, di bawah Easy (89%). Jadi target "win rate naik seiring level" **tidak tercapai** dengan aturan makan yang sekarang. Penyebabnya belum diteliti dengan uji khusus; yang bisa disebut sebagai dugaan:

- **Kedalaman ganjil dan genap.** Depth 1 dan 3 berhenti setelah langkah AI sendiri, depth 2 dan 4 setelah balasan lawan. Dengan aturan makan, hasilnya bisa berbeda cukup jauh antara ganjil dan genap.
- **Greedy selalu makan kalau bisa**, dan hanya pemain Oranye yang bisa makan. Karena AI bergantian jadi Biru dan Oranye, di separuh game AI hanya bisa menang dengan mengurung.
- Fungsi evaluasi BFS/DFS adalah perkiraan, dan pembatasan 12 kandidat jebakan bisa membuang langkah yang ternyata bagus.
- Dengan 100 game, selisih beberapa poin persen masih bisa karena variasi acak, tetapi selisih Medium–Hard (21 poin) terlalu besar untuk dijelaskan hanya dengan itu.

Kalau ditanya, sampaikan angka ini apa adanya sebagai temuan pengujian, dan jangan menyebut satu penyebab sebagai yang pasti.

3. Waktu respons (Tabel 2.2.3, baris 3)

Level	Langkah diukur	Rata-rata	p95	Maksimum
Easy	957	0,2 ms	0,3 ms	10,1 ms
Medium	826	1,2 ms	4,1 ms	8,0 ms
Hard	795	17,2 ms	50,9 ms	92,2 ms
Expert	842	148,2 ms	443,4 ms	696,1 ms

Target "semua level di bawah 4 detik, Easy–Hard di bawah 1 detik" **tercapai jauh**. Angka ini diukur di Node.js dengan kode yang sama dengan Web Worker di browser; di laptop yang lebih lambat angkanya bisa lebih besar, tetapi masih jauh dari 4 detik. *p95* artinya 95% langkah selesai di bawah angka itu.

4. Hal yang belum dibuktikan oleh tes otomatis

- **Playtest** (±10 siswa, kuesioner skala 1–5) adalah uji manusia; hasilnya harus dicatat sendiri oleh tim. Jangan sebut angka playtest yang tidak benar-benar dilakukan.
- Waktu respons di browser bisa dicek langsung di panel **Statistik AI** saat bermain (kolom Waktu).

tests/run.mjs : baris 16–22

```
16  function check(name, cond, detail = '') {
17    if (cond) passes++;
18    else {
19      failures++;
20      console.log(`  FAIL ${name}${detail ? ' :: ' + detail : ''}`);
21    }
22  }
```

13 Tambahan di Luar Proposal: Makan dan Waktu Giliran

Dua aturan ini **tidak ada di proposal**, tetapi ada di game, jadi bisa ditanya saat demo. Bukan bagian dari empat algoritma AI. Versi sekarang juga sudah tidak punya poin, leaderboard, suara, atau animasi; tampilannya dibuat sederhana.

Makan

Aturan (d) di proposal melarang mendarat di kotak lawan. Di kode, mendarat **tepat** di kotak knight musuh justru **memakan** musuh, seperti di catur: game langsung selesai, tidak ada jebakan, dan yang makan menang. Bersebelahan dengan musuh tidak berarti apa-apa; game tetap jalan.

Tempat	Yang dilakukan
`legalJumps()`, `canMove()`	Kotak musuh dihitung sebagai lompatan legal (cek `cells[n] === EMPTY`, bukan `isOpen`).
`isOpen()`	Tetap melarang jebakan di kotak yang ada knight-nya.
`validateMove()`	Langkah makan sah hanya dengan `trap = -1`.
`winner()`	Dua knight di satu kotak berarti yang sedang jalan baru saja dimakan, jadi lawannya menang.
`Game.handleCell()`	Klik di kotak musuh langsung mengakhiri giliran tanpa fase jebakan.
`ai.js`	`generateMoves()` membuat langkah makan sebagai `pack(j, -1)`; `orderMoves()` memeriksanya paling dulu; `ab()` dan `evaluate()` menilai "dimakan" sebagai kalah dan "bisa makan" sebagai menang di ply berikutnya.

tests/run.mjs : baris 85–96

```
85    // Eating: landing exactly on the enemy's square wins at once, with no trap.
86    const eatBlue = createState({ p1: sq('a1'), p2: sq('c2'), first: 0 });
87    check('blue eats: no trap is valid', validateMove(eatBlue, { jump: sq('c2'), trap: -1 }));
88    check('blue eats: a real trap is rejected', !validateMove(eatBlue, { jump: sq('c2'), trap: sq('a7')
}));
89    applyMove(eatBlue, { jump: sq('c2'), trap: -1 });
90    check('blue eats: blue wins', winner(eatBlue) === 0);
91    check('blue eats: both knights share the square', eatBlue.pos[0] === sq('c2') && eatBlue.pos[1] ===
sq('c2'));
92
93    const eatOrange = createState({ p1: sq('e6'), p2: sq('g7'), first: 1 });
94    check('orange eats: no trap is valid', validateMove(eatOrange, { jump: sq('e6'), trap: -1 }));
95    applyMove(eatOrange, { jump: sq('e6'), trap: -1 });
96    check('orange eats: orange wins', winner(eatOrange) === 1);
```

> **Kenapa hanya Oranye yang bisa makan?** Setiap lompatan knight selalu pindah warna kotak (terang ↔ gelap). a1 dan g7 berwarna sama. Biru selalu jalan duluan, jadi pada giliran Biru kedua knight sudah melompat sama banyak dan berdiri di warna yang sama; lompatan Biru pasti mendarat di warna lain, jadi tidak mungkin kena Oranye. Pada giliran Oranye, Biru sudah melompat sekali lebih banyak, jadi warnanya berbeda dan lompatan Oranye bisa mendarat di kotak Biru. Akibatnya Biru hanya bisa menang dengan mengurung. Mengubah posisi awal hanya menukar sisi mana yang bisa makan.

Waktu giliran

Satu giliran manusia maksimal `TURN_SECONDS` (15 detik, di `config.js`) untuk lompat **dan** jebak sekaligus. Sisa waktu tampil di samping status giliran dan menjadi merah saat tinggal 3 detik.

`js/game.js : baris 165–174`

```
165    tickClock() {
166      const left = this.deadline - performance.now();
167      this.ui.timer(Math.max(0, Math.ceil(left / 1000)));
168      if (this.current.kind === 'human') {
169        if (left <= 0) this.timeUp();
170      } else if (this.role === 'host' && left <= -TURN_GRACE_MS) {
171        this.stopClock();
172        this.netQueue = this.netQueue.then(() => this.playForQuietGuest()).catch((err) =>
console.error(err));
173      }
174    }
```

`js/game.js : baris 178–191`

```
178    timeUp() {
179      this.stopClock();
180      this.ui.timeUp();
181      const s = this.state;
182      if (this.phase === 'jump') {
183        const { jump, trap } = randomMove(s);
184        this.handleCell(jump);
185        if (this.phase === 'trap') this.handleCell(trap);
186      } else if (this.phase === 'trap') {
187        const open = [];
188        for (let sq = 0; sq < SIZE; sq++) if (isOpen(s, sq)) open.push(sq);
189        this.handleCell(open[Math.floor(Math.random() * open.length)]);
190      }
191    }
```

- **Waktu habis sebelum lompat:** `randomMove()` memilih langkah acak yang lengkap (lompatan + jebakan). **Sudah lompat:** lompatanmu tetap, hanya jebakannya yang dipilih acak.
- Langkah acak dimasukkan lewat `handleCell()`, sama seperti klik, jadi aturan dan pengiriman online tidak ditulis dua kali.
- **Batal lompat tidak mengulang waktu:** `deadline` tidak berubah.
- **AI tidak memakai penghitung ini.** AI punya batas 4 detik sendiri (bagian Louis).
- **Online:** perangkat masing-masing pemain bertindak saat waktunya habis. Kalau tamu diam lebih dari `TURN_GRACE_MS` (1,5 detik) setelah batas, host memainkan langkah acak untuknya.
- Bagian 6 di `tests/run.mjs` menguji semua hal di atas dengan batas waktu yang dibuat pendek.

Petunjuk di papan

Kotak yang bisa kamu tuju selalu ditandai titik merah. Kotak yang bisa dituju musuh (di fase jebakan) disembunyikan, dan baru tampil bergaris putus-putus setelah tombol **P** ditekan. Setiap game baru mulai dengan petunjuk musuh tersembunyi.

14 Latihan

Latihan 1: Alpha-Beta dengan tangan

MAX punya tiga langkah. Daun (kiri ke kanan) di bawah tiap simpul MIN: A = [4, 8], B = [6, 1, 7], C = [3, 9]. Hitung nilai akar dan daun mana yang dipangkas.

Jawaban: A = 4 (α = 4). B: daun 6 → β = 6, daun 1 → β = 1 ≤ 4 → cutoff, daun 7 dilewati, B ≤ 1. C: daun 3 → β = 3 ≤ 4 → cutoff, daun 9 dilewati. Akar = 4, pilih A. Dua daun dipangkas.

Latihan 2: Packing

Berapa `pack(6, -1)` dan apa hasil `unpack(1999)`?

Jawaban: `6 | (0 << 6) = 6`. 1999 = 31 × 64 + 15 → jump = 15, trap = 31 − 1 = 30.

Latihan 3: Coba di kode

1. Ubah `maxTrapCandidates` dari 12 ke 6 di `config.js`, jalankan `node tests/run.mjs --quick`. Apa yang terjadi pada waktu dan win rate? (Kembalikan setelahnya.)
2. Ubah `orderMinDepth` ke 99 (praktis mematikan pengurutan). Bandingkan kolom "nodes Alpha-Beta" di bagian 2.

15 Pertanyaan Presentasi yang Mungkin Muncul

T: Apa bedanya Minimax dan Alpha-Beta?

J: Minimax memeriksa semua cabang. Alpha-Beta memakai batas α dan β untuk melewati cabang yang terbukti tidak mengubah keputusan. Hasilnya sama, node jauh lebih sedikit: di depth 3 hemat 94,6%.

T: Bagaimana kalian yakin hasilnya sama?

J: Uji di 30 posisi dari seed tetap, depth 1–3: skor dan langkah identik 30/30. Bedanya hanya flag `prune` di fungsi yang sama.

T: Kenapa depth 1 tidak ada penghematan?

J: Di depth 1 semua anak akar langsung dievaluasi; tidak ada simpul MIN tempat β bisa turun di bawah α, jadi tidak ada cutoff.

T: Kenapa kandidat jebakan dibatasi 12? Bukankah itu bisa membuang langkah bagus?

J: Bisa, dan itu trade-off yang kami pilih. Tanpa batas, branching factor bisa ±360 dan Expert tidak akan selesai dalam 4 detik. Jebakan yang efektif hampir selalu di sekitar lawan, jadi ring 1 dan ring 2 dipilih lebih dulu.

T: Apa yang terjadi kalau waktu habis?

J: Pencarian dihentikan dengan exception `ABORT`; AI memakai langkah terbaik dari langkah akar yang sudah selesai dihitung, dan panel menampilkan "batas 4 dtk".

T: Bagaimana AI tahu soal aturan makan?

J: Di tiga tempat: `generateMoves` membuat langkah makan tanpa jebakan, `orderMoves` memeriksanya paling dulu, dan `ab` / `evaluate` menilai posisi "dimakan" sebagai kalah dan "bisa makan" sebagai menang di ply berikutnya. Karena itu Easy (depth 1) pun tidak melompat ke kotak yang bisa dimakan musuh kalau ada pilihan aman.

T: Kenapa skor menang dikurangi ply?

J: Supaya AI memilih kemenangan tercepat dan, kalau kalah, kekalahan terlambat. Tanpa itu, semua kemenangan bernilai sama dan AI bisa "bermain-main".

T: Kenapa win rate melawan greedy tidak naik seiring level?

J: Itu temuan pengujian yang kami laporkan apa adanya: Medium 100%, Hard 79%. Penyebabnya belum diuji khusus; dugaan kami ada di Bab Hasil Pengujian (efek kedalaman ganjil/genap dengan aturan makan, evaluasi yang berupa perkiraan, pembatasan kandidat). Melawan pemain acak semua level 99–100%.

T: Apakah ini memakai machine learning seperti AlphaZero [2]?

J: Tidak. Referensi [2] dipakai sebagai konteks game AI. Kami hanya memakai empat algoritma yang diizinkan: Minimax, Alpha-Beta, BFS, DFS. Tidak ada training.

16 Checklist Sebelum Presentasi

- ☐ Bisa menggambar contoh pohon Alpha-Beta di papan tulis dan menunjuk cutoff-nya.
- ☐ Bisa membuka `js/ai.js` dan menunjuk baris untuk setiap baris pseudocode.
- ☐ Hafal angka: depth 3 → 999.825 vs 54.465 node (hemat 94,6%), 30/30 identik.
- ☐ Sudah mencoba demo Alpha-Beta ON/OFF di Hard dan melihat panel berubah.
- ☐ Sudah menjalankan `node tests/run.mjs --quick` sendiri.
- ☐ Bisa menjelaskan alur satu giliran (Bab 4) tanpa melihat catatan.